DP Sequence

LCS, ED, LPS, DS, SCS

Jakarin Chawachat

ปัญหา Longest Common Subsequence (LCS)

- Longest Common Subsequence (LCS)
  - ลำดับอักขระที่ยาวที่สุดที่พบร่วมกัน (Longest Common Subsequence - LCS) ใช้ในการค้นหาลำดับที่ยาวที่สุดซึ่งปรากฏในลำดับเดียวกันภายในสตริงสองชุด
  - นั่นคือการเปรียบเทียบความคล้ายคลึงของสายอักขระ
  - พบในงาน DNA sequencing, การเปรียบเทียบโค้ด, spell checking ฯลฯ

- ตัวอย่าง
  - ให้ X=$ACDBE$และ Y=$ABCDE$
  - LCS ของ X และ Y คือ $ACDE$

Bruteforce

- สมมติว่า มี sting x[1...m] และ y[1..n] ที่ต้องการหา LCS
  - เราจะตรวจสอบทุกๆ subsequence ของ x ว่า เป็น subsequence ของ y หรือไม่
- วิเคราะห์
  - หากเรามี subsequence อยู่ 1 แบบ การจะเปรียบเทียบว่าทั้ง 2 string มี subsequence ตรงกันไหมใช้เวลาเท่าไร
  - ใช้เวลา O(m+n)
  - กรณี string ยาว n ตัว จะสามารถสร้าง substring ได้ 2^n แบบ
  - ดังนั้น worst case running time ในการเปรียบเทียบทุกรูปแบบคือ $O((m+n)\ 2^n)$

ลองคิดกัน

- ให้ X และ Y เป็นสองสตริง ยาว m และ n ตามลำดับ ต้องการหาความยาวของ **LCS**

หากกำลังพิจารณา string x ถึงตำแหน่งอักขระที่ i

และพิจารณา string y ถึงตำแหน่งอักขระที่ j

สิ่งที่เกิดขึ้นได้มีอะไรบ้าง

- **x[i] = y[j]** หมายถึงกรณีอักขระใน string x ตัวที่ i และอักขระใน string y ตัวที่ j เหมือนกัน
- **x[i] != y[j]** หมายถึงกรณีอักขระใน string x ตัวที่ i และอักขระใน string y ตัวที่ j ไม่เหมือนกัน

- กรณี x[i] = y[j]
- แล้วเราเราได้อะไร และไปถามใครต่อ
- เราได้ความยาวเพิ่มขึ้นเพราะว่าเหมือนกัน

จากนั้นไปถามstring x ถึงตำแหน่งอักขระที่ i-1

และพิจารณา string y ถึงตำแหน่งอักขระที่ j-1

- กรณี x[i] **!=** y[j]
- แล้วเราเราได้อะไร และไปถามใครต่อ
- เราไม่ได้ความยาวเพิ่มขึ้น

จากนั้นไปถามstring x ถึงตำแหน่งอักขระที่ i-1 และ

และพิจารณา string y ถึงตำแหน่งอักขระที่ j

กับ ไปถามstring x ถึงตำแหน่งอักขระที่ i และ

และพิจารณา string y ถึงตำแหน่งอักขระที่ j-1

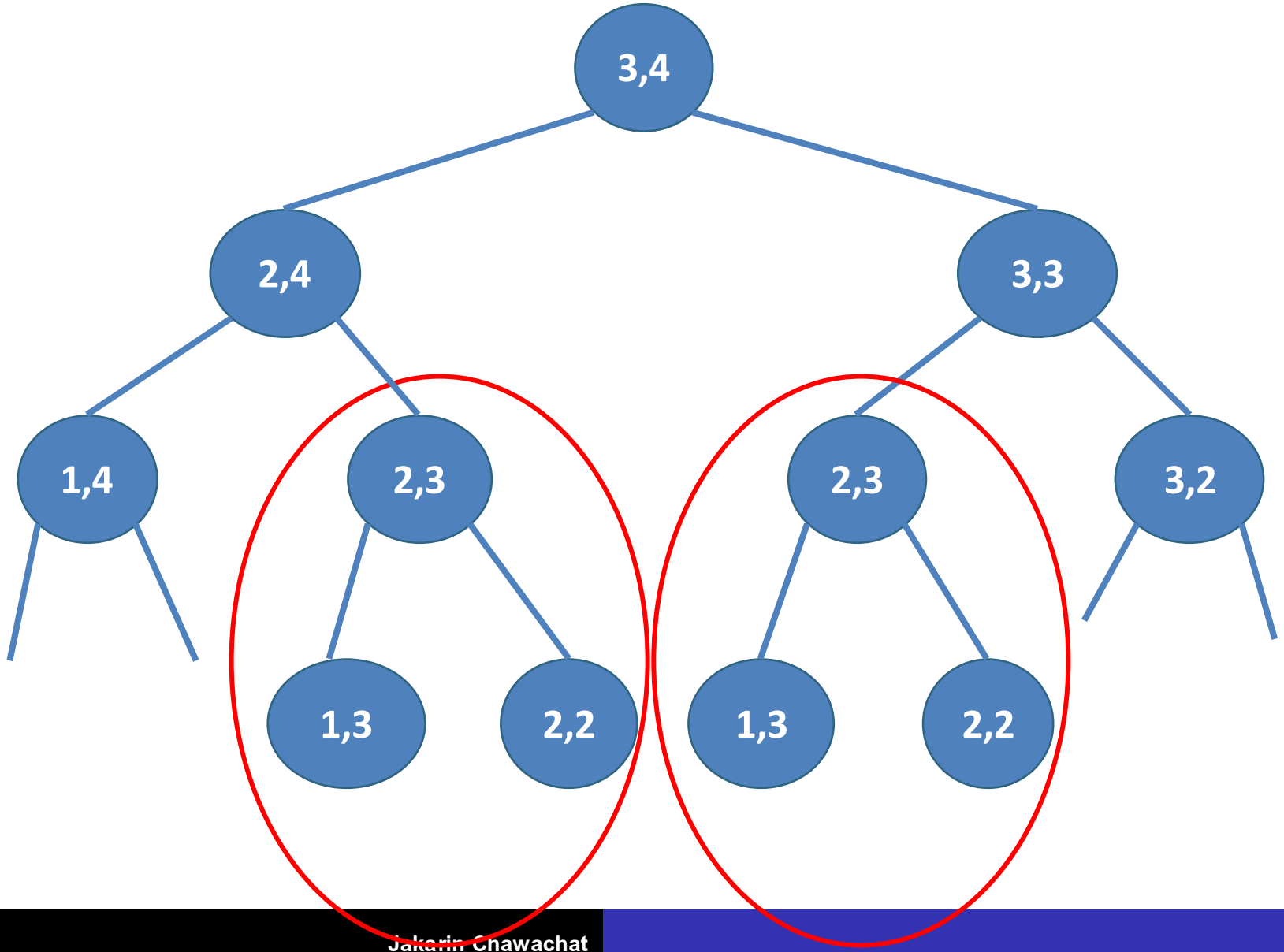
- ว่าแบบไหนดีกว่ากัน

$$LCS(i,j) = \begin{cases} 0, & i = 0, j = 0 \\ 1 + LCS(i-1, j-1), & X[i] = Y[j] \\ \max(LCS(i-1,j), LCS(i,j-1)), & X[i] \neq Y[j] \end{cases}$$

Recursive

```
int LCS(string X, string Y, int i, int j) {
    if (i == 0 || j == 0) return 0;
    if (X[i-1] == Y[j-1])
        return 1 + LCS(X, Y, i-1, j-1);
    return max(LCS(X, Y, i-1, j), LCS(X, Y, i, j-1));
}
```

ตัวอย่าง overlapping subproblem



Memoization

```
int dp[1001][1001];
int LCS(string X, string Y, int i, int j) {
    if (i == 0 || j == 0) return 0;
    if (dp[i][j] != -1) return dp[i][j];

    if (X[i-1] == Y[j-1])
        return dp[i][j] = 1 + LCS(X, Y, i-1, j-1);
    return dp[i][j] = max(LCS(X, Y, i-1, j), LCS(X, Y, i, j-1));
}
```

Tabulation

```
int LCS(string X, string Y) {
    int m = X.length(), n = Y.length();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (X[i-1] == Y[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n];
}
```

ปัญหา Edit Distance (Levenshtein Distance)

- ใช้ใน spell checking, DNA sequencing, และ text similarity
- คำนวณจำนวนการแก้ไขที่น้อยที่สุดระหว่างสองสตริงโดยใช้ Insert, Delete, Replace
- ตัวอย่าง
  - "kitten" ⟶ "sitting" ต้องใช้ 3 การแก้ไข:
    - Replace 'k' ⟶ 's'
    - Replace 'e' ⟶ 'i'
    - Append 'g'

ลองคิดกัน

- ให้สตริง X และ Y คำนวณจำนวนการแก้ไขขั้นต่ำเพื่อแปลง X เป็น Y
- หากพิจารณา X[i] และ Y[j]
  - กรณีที่เหมือนกัน
    - ก็ไม่เพิ่ม cost แล้วไปพิจารณา X[i-1] และ Y[j-1]
  - กรณีที่ต่างกัน
    - ก็เลือกว่าเสีย cost แล้วพิจารณาทุกแบบที่เหลือที่ให้ cost น้อยสุด
    - ดังนั้นเสีย cost 1
    - $\min(ED(i-1,j), ED(i,j-1), ED(i-1,j-1))$

- Base case
- กรณีที่มีแต่ i หรือมีแต่ j
  - ถ้ามีแต่ i ก็เสียค่าเปลี่ยนเท่ากับความยาว i
  - ถ้ามีแต่ j ก็เสียค่าเปลี่ยน เท่ากับความยาว j

$$ED(i,j) = \begin{cases} i, & j = 0 \\ j, & i = 0 \\ ED(i-1, j-1), & X[i] = Y[j] \\ 1 + \min(ED(i-1,j), ED(i,j-1), ED(i-1,j-1)), & X[i] \neq Y[j] \end{cases}$$

Recursive

```
int EditDistance(string X, string Y, int i, int j) {
    if (i == 0) return j;
    if (j == 0) return i;
    if (X[i-1] == Y[j-1])
        return EditDistance(X, Y, i-1, j-1);

    return 1 + min({
        EditDistance(X, Y, i-1, j),     // Delete
        EditDistance(X, Y, i, j-1),     // Insert
        EditDistance(X, Y, i-1, j-1)    // Replace
    });
}
```

Time Complexity: $O(3^{m+n})$

Memoization

```
int dp[1001][1001];
int EditDistance(string X, string Y, int i, int j) {
    if (i == 0) return j;
    if (j == 0) return i;
    if (dp[i][j] != -1) return dp[i][j];

    if (X[i-1] == Y[j-1])
        return dp[i][j] = EditDistance(X, Y, i-1, j-1);

    return dp[i][j] = 1 + min({
        EditDistance(X, Y, i-1, j),
        EditDistance(X, Y, i, j-1),
        EditDistance(X, Y, i-1, j-1)
    });
}
```

Tabulation

```
int EditDistance(string X, string Y) {
    int m = X.length(), n = Y.length();
    vector<vector<int>> dp(m+1, vector<int>(n+1));
    for (int i = 0; i <= m; i++)
        for (int j = 0; j <= n; j++)
            dp[i][j] = (i == 0) ? j : (j == 0) ? i : 0;

    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            dp[i][j] = (X[i-1] == Y[j-1]) ? dp[i-1][j-1] :
                        1 + min({dp[i-1][j], dp[i][j-1],
dp[i-1][j-1]});

    return dp[m][n];
}
```

ปัญหา Longest Palindromic Subsequence (LPS)

- ใช้ในการวิเคราะห์ DNA, การเข้ารหัสข้อมูล และการตรวจสอบความสมมาตรของสตริง
- ต่างจาก Longest Palindromic Substring (LPSs) ตรงที่ subsequence ไม่จำเป็นต้องต่อเนื่อง
- ตัวอย่างให้ X="$BBABCBCAB$
  - LPS ของ X คือ "BABCBAB" หรือ "BACBCAB"
  - ความยาวของ LPS คือ 7
  - **LPS:** LeetCode #516

นิยามปัญหา

- ให้สตริง X ความยาว n ต้องการหาความยาวของ **Longest Palindromic Subsequence**
- เวลาดู palindrome คือดูหัวท้ายเหมือนกันไหม
  - สมมติว่าหัวคือ i ท้ายคือ j
  - ถ้าเหมือนก็ได้ความยาวเท่าไร แล้วไปถามใคร
  - ถ้าไม่เหมือนได้ความยาวเท่าไร แล้วไปถามใคร

ลองคิดกัน

- ถ้าหัวท้ายเหมือนกัน
  - ได้ความยาวเพิ่ม 2
  - ไปถาม $LPS(i+1, j-1)$
- ถ้าไม่เหมือนกัน
  - ไม่ได้ความยาวเพิ่ม
  - ไปถามทุกแบบ นั่นคือ $\max(LPS(i+1, j), LPS(i, j-1))$

$$LPS(i,j) = \begin{cases} 1, & i = j \\ 0, & i > j \\ 2 + LPS(i+1, j-1), & X[i] = Y[j] \\ \max(LPS(i+1,j), LPS(i,j-1)), & X[i] \neq Y[j] \end{cases}$$

Recursive

```
int LPS(string X, int i, int j) {
    if (i == j) return 1;
    if (i > j) return 0;

    if (X[i] == X[j])
        return 2 + LPS(X, i+1, j-1);

    return max(LPS(X, i+1, j), LPS(X, i, j-1));
}
```

Time Complexity: $O(2^n)$

Memoization

```
int dp[1001][1001];

int LPS(string X, int i, int j) {
    if (i == j) return 1;
    if (i > j) return 0;
    if (dp[i][j] != -1) return dp[i][j];

    if (X[i] == X[j])
        return dp[i][j] = 2 + LPS(X, i+1, j-1);

    return dp[i][j] = max(LPS(X, i+1, j), LPS(X, i, j-1));
}
```

Tabulation

```
int LPS(string X) {
    int n = X.length();
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++) dp[i][i] = 1;
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n-len; i++) {
            int j = i + len - 1;
            if (X[i] == X[j])
                dp[i][j] = 2 + dp[i+1][j-1];
            else
                dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
        }
    }
    return dp[0][n-1];
}
```

ปัญหา Distinct Subsequences

- ให้สตริง s และ t จงคืนค่าจำนวนลำดับอักขระย่อยที่แตกต่างกันของ sซึ่งมีค่าเท่ากับ t
- ใช้ในการวิเคราะห์ข้อมูล, text pattern matching, และการตรวจสอบ sub-patterns

ตัวอย่าง

ให้ X="rabbbit", Y="rabbit"

- จำนวนรูปแบบที่ X สามารถสร้าง Y ได้คือ **3**
  - **rabb**bit
  - **ra**b**bbit**
  - **rab**b**bit**
- LeetCode #115

ลองคิดกัน

- ให้สตริง X และ Y ต้องการหาจำนวน **subsequences** ของ X ที่เป็น Y
- ถ้าพิจารณา X[i] และ Y[j] **ข้อนี้ดูดี ๆ**

- กรณีที่ไม่เหมือนกัน
  - เลื่อน i แต่ j เท่าเดิมเพราะว่ายังไม่ตรง นั่นคือเลื่อนไปดู X[i-1] และ Y[j]
- กรณีที่เหมือนกัน
  - เลื่อนไปดู X[i-1] และ Y[j-1] แค่นี้พอไหม ถ้าแค่นี้จะได้ค่าว่าเจอแบบนึง
  - เราอยากนับทุกแบบ เราก็ต้องสมมติว่าไม่ใช้ตัวนี้ด้วย X[i-1] และ Y[j]
  - แล้วทำต่อไปเรื่อยๆ

- แล้ว basecase หละ
- ถ้า j = 0 คือแปลงได้หมดแล้ว จะได้ 1 แบบ
- ถ้า i = 0 คือต้นทางหมดก่อน จะได้ 0 แบบ

$$DS(i,j) = \begin{cases} 1, & j = 0 \\ 0, & i = 0 \\ DS(i-1,j) + DS(i-1,j-1), & X[i] = Y[j] \\ DS(i-1,j), & X[i] \neq Y[j] \end{cases}$$

Recursive

```
int DistinctSubsequences(string X, string Y, int i, int j) {
    if (j == 0) return 1;
    if (i == 0) return 0;

    if (X[i-1] == Y[j-1])
        return DistinctSubsequences(X, Y, i-1, j) +
               DistinctSubsequences(X, Y, i-1, j-1);

    return DistinctSubsequences(X, Y, i-1, j);
}
```

```
int dp[1001][1001];

int DistinctSubsequences(string X, string Y, int i, int j) {
    if (j == 0) return 1;
    if (i == 0) return 0;
    if (dp[i][j] != -1) return dp[i][j];

    if (X[i-1] == Y[j-1])
        return dp[i][j] = DistinctSubsequences(X, Y, i-1, j)
                        + DistinctSubsequences(X, Y, i-1, j-1);

    return dp[i][j] = DistinctSubsequences(X, Y, i-1, j);
}
```

```
int DistinctSubsequences(string X, string Y) {
    int m = X.length(), n = Y.length();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for (int i = 0; i <= m; i++) dp[i][0] = 1;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (X[i-1] == Y[j-1])
                dp[i][j] = dp[i-1][j] + dp[i-1][j-1];
            else
                dp[i][j] = dp[i-1][j];
        }
    }
    return dp[m][n];
}
```

ปัญหา Shortest Common Supersequence (SCS)

- ใช้ใน การรวม DNA sequences, การรวมเอกสาร, และ การบีบอัดข้อมูล
- เป็นปัญหาที่คล้าย LCS แต่ต้องการหาสตริงที่สั้นที่สุดที่มีทั้งสองสตริงเป็น subsequences

ตัวอย่าง

ให้ X="AGGTAB", Y="GXTXAYB"

- **SCS** ที่สั้นที่สุดคือ **"AGGXTXAYB"**
- ความยาวของ SCS คือ **9**

ลองคิดกัน

- ให้สตริง X และ Y ต้องการหาความยาวของ **Shortest Common Supersequence**
- ถ้าพิจารณา X[i] และ Y[j]
  - **ถ้าเหมือนกัน**
    - ก็จะได้ความยาว 1 หน่วย แล้วก็ไปพิจารณา X[i-1] และ Y[j-1]
  - **ถ้าไม่เหมือนกัน**
    - ก็จะได้ความยาว 1 หน่วยอยู่ดีเพราะว่าเลือกตัวนึงมาเติม แล้วก็ไปพิจารณา
      - X[i-1] และ Y[j]
      - X[i] และ Y[j-1]
      - ว่าแบบไหนได้น้อยกว่ากัน

- basecase
- ถ้า i = 0 นั่นคือไม่เหลือแล้ว ก็จะได้ความยาวของ j ที่เหลือ
- ถ้า j = 0 นั่นคือไม่เหลือแล้ว ก็จะได้ความยาวของ i ที่เหลือ

$$SCS(i,j) = \begin{cases} i, & j = 0 \\ j, & i = 0 \\ 1 + SCS(i-1, j-1), & X[i] = Y[j] \\ 1 + \min(SCS(i-1,j), SCS(i,j-1)), & X[i] \neq Y[j] \end{cases}$$

```
int SCS(string X, string Y, int i, int j) {
    if (i == 0) return j;
    if (j == 0) return i;

    if (X[i-1] == Y[j-1])
        return 1 + SCS(X, Y, i-1, j-1);

    return 1 + min(SCS(X, Y, i-1, j), SCS(X, Y, i, j-1));
}
```

```
int dp[1001][1001];

int SCS(string X, string Y, int i, int j) {
    if (i == 0) return j;
    if (j == 0) return i;
    if (dp[i][j] != -1) return dp[i][j];

    if (X[i-1] == Y[j-1])
        return dp[i][j] = 1 + SCS(X, Y, i-1, j-1);

    return dp[i][j]= 1+min(SCS(X, Y, i-1, j),SCS(X, Y, i, j-1));
}
```

```
int SCS(string X, string Y) {
    int m = X.length(), n = Y.length();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (X[i-1] == Y[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n];
}
```